

ELEC210K Computer Programming

Short Coding Practice Solutions

Lecture 6 Quiz: Procedural Programming & Functions

Solution for Question 1:

```
int findMax(int num1, int num2, int num3) {  
    int maxVal = num1;  
    if (num2 > maxVal) {  
        maxVal = num2;  
    }  
    if (num3 > maxVal) {  
        maxVal = num3;  
    }  
    return maxVal;  
}
```

Solution for Question 2:

```
double computePower(double base, int exponent);
```

Solution for Question 3:

```
void printPattern(int n) {  
    for (int i = 0; i < n; i++) {  
        cout << "*";  
    }  
    cout << endl;  
}
```

Solution for Question 4:

Inside function: 15

Inside main: 5

Explanation: Under pass-by-value, a copy of 'num' is sent to the function. Changes made to 'x' inside the function scope do not affect 'num' in main.

Solution for Question 5:

```
double calculateHypotenuse(double sideA, double sideB) {  
    return sqrt(pow(sideA, 2) + pow(sideB, 2));  
    // Alternatively: return sqrt((sideA * sideA) + (sideB * sideB));  
}
```

Lecture 7 Quiz: Introduction to Classes & Objects

Solution for Question 1:

```
class Circle {  
private: // or protected  
    double radius;  
public:  
    Circle(double r) {  
        radius = r;  
    }  
};
```

Solution for Question 2:

```
: hours(h), minutes(m), seconds(s)
```

Solution for Question 3:

```
double getRadius() const {  
    return radius;
```

```
}
```

Solution for Question 4:

```
void setRadius(double r) {  
    if (r > 0) {  
        radius = r;  
    }  
}
```

Solution for Question 5:

```
int main() {  
    Time lunchTime(12, 30, 0); // Object instantiation  
    lunchTime.print();          // Invoking the member function  
    return 0;  
}
```

Lecture 9 Quiz: References & Parameter Passing

Solution for Question 1:

```
int& rScore = score;
```

Solution for Question 2:

```
void swapValues(int& first, int& second) {  
    int temp = first;  
    first = second;  
    second = temp;  
}
```

Solution for Question 3:

```
void printMessage(const string& str) {  
    cout << str << endl;  
}
```

Solution for Question 4:

```
int& getLarger(int& a, int& b) {  
    if (a > b) {  
        return a;  
    } else {  
        return b;  
    }  
}
```

Solution for Question 5:

```
a: 30  
b: 15
```

Explanation: 'int& ref = a;' binds ref permanently to a.
The expression 'ref = b;' assigns the VALUE of b (15) into a.
Then 'ref = 30;' explicitly updates a to 30. Variable b is completely unaffected.

Lecture 11 Quiz: Inheritance & Class Relationships

Solution for Question 1:

```
class Student : public Person
```

Solution for Question 2:

```
: Person(nm), gpa(g)
```

Solution for Question 3:

```
void print() const {  
    Person::print(); // 1. Calls the base class print function  
    cout << "GPA: " << gpa << endl; // 2. Prints the unique derived attribute  
}
```

Solution for Question 4:

```
void changeName(string newName) {  
    name = newName; // Direct access because 'name' is protected in Base class  
}
```

Solution for Question 5:

```
Base Con  
Derived Con  
Derived Des  
Base Des
```

Lecture 12 Quiz: Polymorphism & Abstract Classes

Solution for Question 1:

```
virtual double getArea() const = 0;
```

Solution for Question 2:

```
double getArea() const override {  
    return width * height;  
}
```

Solution for Question 3:

```
virtual ~Base() {  
    cout << "Base destructor" << endl;
```

```
}
```

Solution for Question 4:

```
void printShapeArea(const Shape* shapePtr) {  
    if (shapePtr != nullptr) {  
        cout << "The area is: " << shapePtr->getArea() << endl;  
    }  
}
```

Solution for Question 5:

```
class Sub1 : virtual public Person {  
    // fields/methods for Sub1  
};  
  
class Sub2 : virtual public Person {  
    // fields/methods for Sub2  
};
```

Lecture 13 Quiz: Object Roles & Operator Overloading

Solution for Question 1:

```
void print() const {  
    cout << numer << "/" << denom << endl;  
}
```

Solution for Question 2:

```
Fraction& operator++() {  
    numer = numer + denom; // Increment fraction value by 1  
    return *this;          // Return the reference to the host object  
}
```

Solution for Question 3:

```
const Fraction operator++(int) {  
    Fraction temp(number, denom); // 1. Save the original state  
    ++(*this);                    // 2. Call the pre-increment operator  
    return temp;                  // 3. Return the saved original state  
}
```

Solution for Question 4:

```
const Fraction operator+(const Fraction& left, const Fraction& right) {  
    int new_numer = left.numer * right.denom + right.numer * left.denom;  
    int new_denom = left.denom * right.denom;  
    return Fraction(new_numer, new_denom);  
}
```

Solution for Question 5:

```
Fraction& operator=(const Fraction& right) {  
    // Check for self-assignment (e.g., f1 = f1)  
    if (this != &right) {  
        numer = right.numer;  
        denom = right.denom;  
    }  
    return *this; // Return reference to host object  
}
```